
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 0: Project Description

Project Type: Code Generation

September 16, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Problem Statement	1
2	Solution	1
3	Source Language	1
4	Generated Code	1
5	Metamodels	2
6	Implementation Approach	2

1 Problem Statement

A Kubernetes cluster typically runs a large number of services. Each service needs a workload, service account, network policies, ingress routes, monitoring rules and a disruption budget. These resources repeat the same facts in different schemas. For example, a port appears in four files, while one dependency sets both an address and an egress rule.

These facts are spread across several places. The path to one service can span several repositories. Manually maintained copies can drift apart even when each file is valid. The cluster accepts the files, but an application may no longer reach its database or its metrics may not be collected.

Service dependencies also change over time. Changing one dependency currently requires edits to every dependent service. A model can resolve dependencies automatically and avoid these repeated edits. The dependency is changed once, and every related service receives the new values. This project will generate a consistent Kubernetes deployment set from one abstract application model.

2 Solution

The domain-specific language records processes, ports, dependencies, routes, storage, secrets, health and operational requirements once. It does not name Kubernetes resources or vendor fields.

The project objective is to define this abstract model and the transformations that turn it into a complete set of deployment YAML and JSON files.

For each release the generator is given a closed set of inputs, every one of them carried by digest: each project's own model with its per-process environment files, one platform document stating what the estate offers, a contract publishing what each machine can hold, an image lock mapping image names to pinned digests, and a snapshot of the few cluster facts nothing else can supply, such as which machine already holds a volume. The generator reads these and nothing else. It does not read live cluster state and it does not access secret values.

A model-to-model transformation turns these inputs into a *Resolved Deployment*. This complete, non-text model contains exact resource names, namespaces, paths, image digests, replicas, resources, probes, ports, addresses, policy peers, routes, secret references and monitor targets. Renderers only turn this model into files and make no deployment decisions.

The model-to-text step generates one *Deliverable Set* for the estate: Kubernetes YAML, policy JSON and the index files that group them, laid out per project and application by a plan the resolved model carries, so every file's path is a decision recorded in the model rather than one a generator makes while writing. Six generators produce it, each run once over the whole composed input rather than once per application, which is what lets a value one application derives from another be computed in one place.

How that set reaches a cluster is deliberately outside this project. The model states what should be true and what must be applied before what; whether a pipeline applies it directly or a cluster-side reconciler consumes the pipeline's published output is a separate decision, and the specification is written so that either can be chosen without changing it. Platform facts come from one central, versioned source, so every repository resolves against the same platform definition.

3 Source Language

One source file represents a project and its applications. Table 1 summarises the language.

The model describes application needs rather than Kubernetes kinds, proxy fields or machine names. The model-to-model transformation adds the platform-specific details before the final files are generated.

4 Generated Code

The generators create one versioned Deliverable Set for the estate, laid out per project and application by the path plan the resolved model carries. Table 2 lists its resources and how they are derived.

The transformation computes limits, startup probes, route precedence and policy peers. For `auth`, 96 declared lines produce eleven files of about 620 lines. The repository contains hand-written expected files, which are used to check the generated result. The tree carries one index entry point at its root, so the whole estate is applied through a single reference rather than one per application.

Concern	Model content
Ownership and release	Who owns the project and which processes are released together as one application.
Processes	Which containerised processes run and whether they are one-time or long-running.
Placement and scale	A process's CPU, memory, allowed machines and number of running instances.
Exposure and routing	An application's web address, who may reach it and which process handles each URL path.
Connections	The ports a process offers and the applications it must reach.
Health and cutover	Checks for process health and readiness, and how a new version replaces an old one.
Observability	Where the platform collects metrics and how urgently a failure raises an alert.
Storage and writes	Where a process stores data, how it is recovered and which folders it may access.
Secrets	Which vault secrets a process may read and how it receives them.

Table 1: Concerns Defined by the Source Language

Output	How it is derived
Namespace and indexes	A named cluster partition for the project and index files listing what to deploy.
Workload resources	Definitions that run processes, give each an identity and keep enough instances available during maintenance.
NetworkPolicy	Firewall rules that allow required process connections and block all others.
IngressRoute	Web proxy rules that send each address and path to the correct process.
Monitoring and alerts	Instructions for collecting metrics and conditions for raising alerts.
Secret resources	Vault permissions granting each process only its allowed secrets.

Table 2: Generated Resources and Their Derivation

5 Metamodels

The pipeline in Section 2 uses two metamodels:

1. **Project Intent**, the source metamodel: *Project, Application, Process, Exposure, Route, Placement, Probe, Dependency, Secret Grant* and *Volume*.
2. **Resolved Deployment**, the target code metamodel: concrete, typed Kubernetes and extension resources with every derived value, relationship, identity and output path.

Acceleo turns a Resolved Deployment model into the Deliverable Set of YAML and JSON files. UML class diagrams document both metamodels, which are defined in Ecore with OCL constraints. The production implementation represents the same structures with TypeScript types and `zod` runtime schemas, as described in Section 6.

6 Implementation Approach

The production implementation uses TypeScript and `zod`. It works with the Node.js pipelines used across the JorisJonkers-dev repositories and maps YAML and JSON directly to typed values. `zod` provides runtime validation, inferred static types and an exported JSON Schema for editor support.

The model-driven implementation defines both metamodels in Ecore and their constraints in OCL. Xtext parses the source language, QVT-OM transforms a Project Intent model into a Resolved Deployment model, and Acceleo generates the Deliverable Set as YAML and JSON. This implementation will show how metamodels and transformations express the same deployment rules as the production pipeline.

Both implementations will use the same examples and expected outputs. They will be compared on validation, dependency resolution, errors and generated resources. The EMF tools will run without an IDE through Maven. The Xtext-generated editor will be configured to run the OCL validators and report constraint violations during editing.